

Compiler Pipeline Comparison

Classic C - Sakum Full - WASM-Only

This document presents three compilation pipelines that share a common front-end (stages 1-12). The user selects a path at the branch point. All pipelines end at a sandbox gate, then a production gate (ask_patch) that writes to update.md on approval.

Legend

 Source / Output

 Front-end (shared)

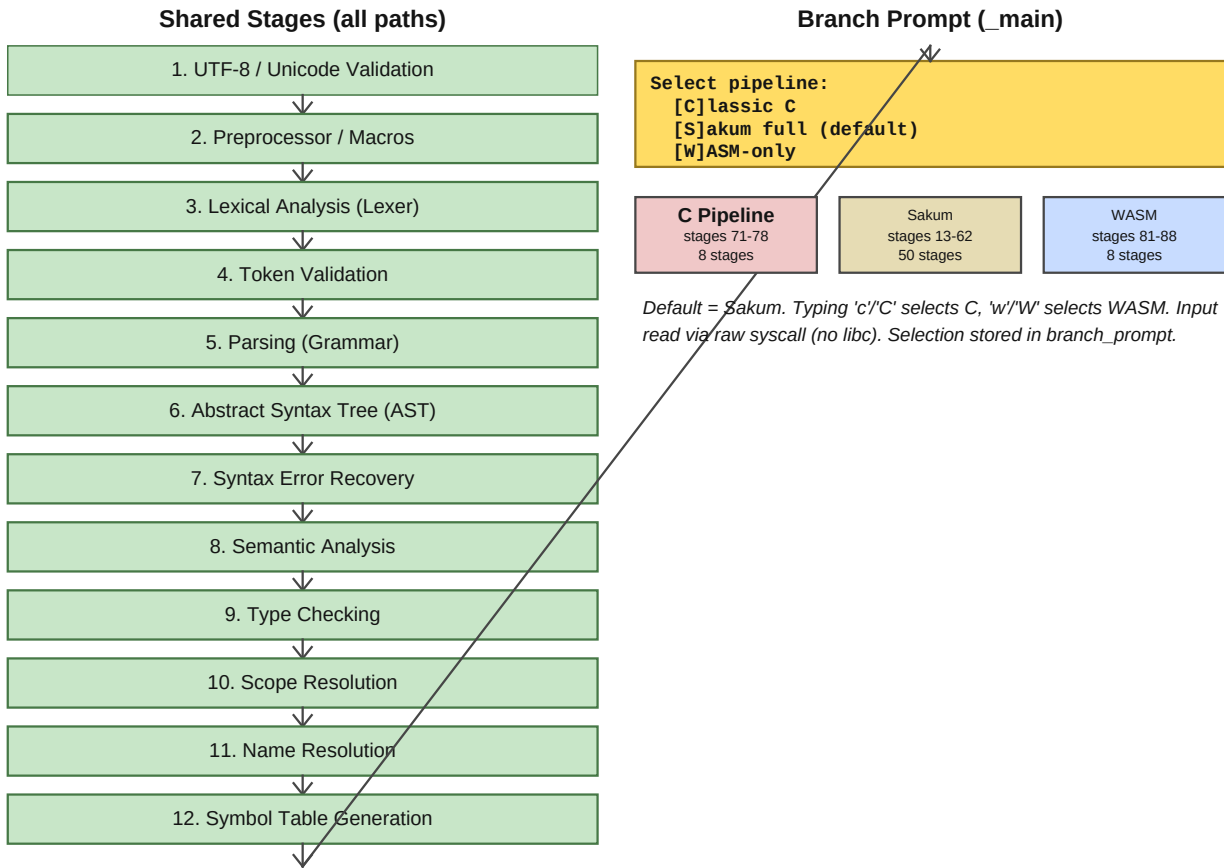
 IR / Analysis

 Codegen / Backend

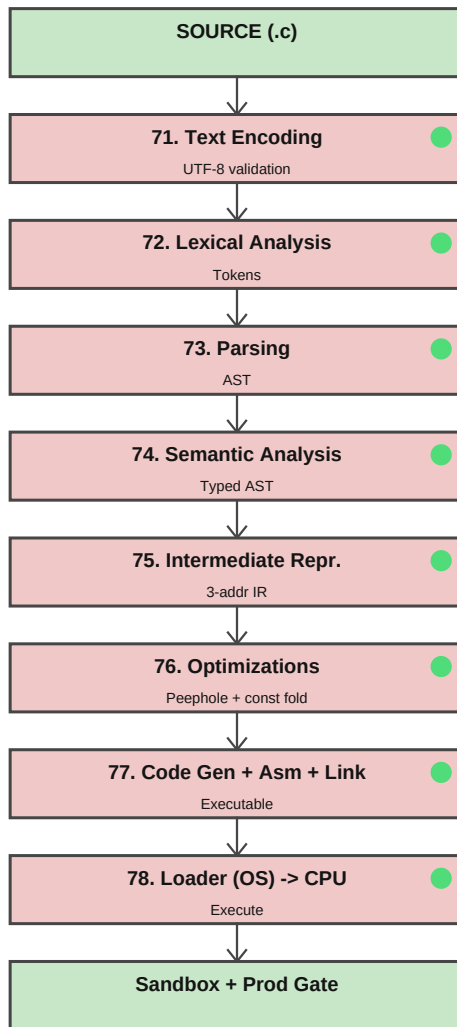
 Runtime / Loader

 Gate / Decision

Branch Point: Shared Front-End (Stages 1-12)

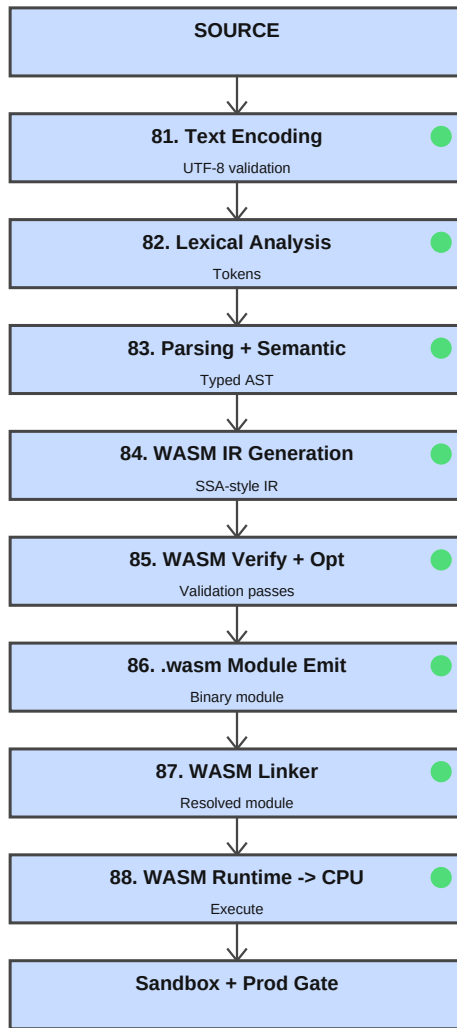


Classic C Pipeline (stages 71-78)



WASM-Only Pipeline (stages 81-88)

A lightweight path that skips native codegen and VM bytecode, emitting only a .wasm module. Best for browser/edge deployment with minimal toolchain.



Sakum Full Pipeline (62 stages)

Grouped by phase: Front-End -> Analysis -> Backends -> Runtime

FRONT-END (shared 1-12)	ANALYSIS 13-30	BACKENDS 31-47	RUNTIME 48-62
1 Unicode Validation	13 Ownership	31 Mach IR	48 Universal Link
2 Preprocessor	14 Borrow/Mem	32 Reg Alloc	49 Lib Resolve
3 Lexer	15 Generics	33 Sched	50 Reloc
4 Token Validation	16 CTFE	34 Asm Emit	51 Exec/Lib
5 Parsing	17 Const Fold	35 Obj File	52 Bin Validate
6 AST	18 HIR	36 VM Bytecode	53 Size Opt
7 Error Recovery	19 HIR Verify	37 VM Verify	54 Debug Sym
8 Semantic	20 HIR Opt	38 VM Opt	55 Package
9 Type Checking	21 MIR/SSA	39 Sanskrit BC	56 Code Sign
10 Scope	22 CFG	40 VM Package	57 Prod Security
11 Name Resolution	23 Dataflow	41 VM Exec	58 OS Loader
12 Symbol Table	24 Alias/Escape	42 WASM IR	59 CRT Init
	25 Mem Opt	43 WASM Verify	60 Mem Layout
	26 Security	44 WASM Opt	61 Fetch/Decode/Exec
	27 DCE/Inline	45 .wasm Emit	62 Sandbox
	28 Loop/Vec	46 WASM Link	
	29 LIR	47 WASM Binary	
	30 Backend Check		

Stage Trace Sample (real /tmp/sakum_pipe output)

```
[1] OK Unicode Validation ... utf-8 byte sequences validated
[5] OK Parsing ... grammar rules applied to token stream
[6] OK Abstract Syntax Tree ... tree built from parse
[9] OK Type Checking ... all types resolved and consistent
[11] OK Name Resolution ... names bound to declarations
[12] OK Symbol Table Generation ... global symbol table built
Select pipeline: [C]lassic C [S]akum full (default) [W]ASM-only
[13] OK Ownership Analysis ... lifetimes assigned
[14] OK Borrow Checking ... no aliasing violations
[18] OK High-Level IR ... HIR constructed
[21] OK Mid-Level IR ... SSA form built
[29] OK Low-Level IR ... LIR lowered
[34] OK Assembly Emission ... asm text written
[42] OK WASM IR Generation ... wasm ir emitted
[56] OK Code Signing ... binary signed
[61] OK CPU Fetch/Decode/Execute ... instructions retired
[62] OK Sandbox ... isolated execution passed
Sandbox passed. Patch to production? [y/N]
Production gate approved -> update.md written
```

Each line is emitted by trace_stage in assembly/sakum_pipe.s. uito() converts the stage number to decimal; status OK/FAIL is appended via strcat; rdi is updated after each strcat so the '...' separator and note render correctly.

Comparison & Decision Guide

Aspect	C	Sakum	WASM-Only
Stages	71-78 (8)	13-62 (50)	81-88 (8)
IR	Single 3-addr	HIR->MIR->LIR	WASM IR (SSA)
Opt	Peephole	Full CFG/DF/Vec	Verify + Opt
Mem Safety	None	Ownership+Borrow	WASM sandbox
Backends	Native x64	Native+VM+WASM	WASM only
Security	None	Sign+Verify	WASM verify
CTFE	No	Yes	Limited
Build Speed	Fast	Slower	Fast
Deploy	Desktop	Desktop/Edge	Browser/Edge
Footprint	Small	Larger	Small

When to choose:

- C: speed, compiling C, learning, single native target, no security needs.
- Sakum: memory safety, CTFE, multi-backend, production security, Sakum Lang.
- WASM-Only: browser/edge deploy, small footprint, fast build, WASM sandbox.